Trabajo Práctico N.º 2

Diseño de Clases

Indicaciones:

- ***i.*** *Al comenzar el TP de debe clonar el repositorio correspondiente y crear la rama local con su nombre.*
- ***ii.*** *Para cada ejercicio se debe crear un Proyecto con el nombre en mayúsculas TP2_EJX.*
- ***iii.*** *Para versionar el trabajo local se debe usar "commit" y para enviar al repositorio remoto usar "push"*
- ***iv.*** *En el método "static void main()" se deben crear las instancias de las clases diseñadas para invocar sus métodos y comprobar su funcionamiento.*
- ***v.*** *En todas las clases, de definen los Constructores, los métodos Getters() y Setters() y los necesarios según corresponda*

Ejercicio 1. Implementar la clase **Fecha** con atributos para el mes, el día y el año. Cada objeto de esta clase representa una fecha, que almacena el día, mes y año como enteros. Se deben incluir los siguientes métodos:

Fecha
- dia: int - mes: int - año: int
+ Fecha() + Fecha(d: int, m: int, a: int) + getDia(): int + getMes(): int + getAño(): int + setDia(d: int): void + setMes(m: int): void + setAño(a: int): void + reiniciar(d: int, m: int, a: int): void + adelantar(d: int): void + imprimir(): void - valida(): boolean - normalizar(): void - dias(mes: int): int + diferenciaEnDias(f: Fecha): int + comparar(f: Fecha): int + clonar(): Fecha + toString(): String + formatear(fmt: String): String + static esBisiesto(a: int): boolean + static hoy(): Fecha

- ☐ **Constructor** por defecto y **Constructor** con parámetros
- ☐ Métodos de acceso (**getters** y **setters**)
- ☐ **reiniciar(int d, int m, int a)** para reiniciar la fecha de un objeto existente
- ☐ **adelantar(int d)** para avanzar una cantidad de días
- ☐ **imprimir()** para mostrar la fecha
- ☐ **valida()** que determine si es una fecha correcta
- ☐ **normalizar()** que corrija los valores para que estén en rangos correctos
- ☐ **dias(int mes)** que devuelve el número de días de cada mes
- ☐ Método estático **esBisiesto(int año)** que determine si un año es bisiesto

Agregar también los siguientes métodos adicionales:

- ☐ **toString()** para representación en formato DD/MM/AAAA
- ☐ **clonar()** para crear una copia de la fecha
- ☐ **comparar(Fecha otra)** para comparar fechas
- ☐ **diferenciaEnDias(Fecha otra)** para calcular días entre fechas
- ☐ **formatear(String formato)** para personalizar la presentación
- ☐ **estático hoy()** que devuelve la fecha actual

Nota: un año es bisiesto si es divisible por 400, o si es divisible por 4 pero no por 100. Por ejemplo, los años 1992 y 2000 son años bisiestos mientras que 1997 y 1800 no son bisiestos.

En el método **main**, instanciar objetos de la clase Fecha para calcular la edad de una persona, comparar dos fechas, calcular los días restantes para un eventos y otros ejemplos de uso.

Ejercicio 2. Desarrollar una clase llamada **Complejo** que permita representar y operar con números complejos. La clase debe tener atributos para la parte real e imaginaria e implementar las operaciones básicas (suma, resta, multiplicación y división) como:

- Métodos de **instancia** que modifican el objeto actual.
- Métodos **estáticos** que reciben dos objetos Complejo y devuelven un nuevo objeto Complejo.

$c \leftarrow c + a$	$c \leftarrow a + b$
$c \leftarrow c - a$	$c \leftarrow a - b$
$c \leftarrow c \times a$	$c \leftarrow a \times b$
$c \leftarrow c \div a$	$c \leftarrow a \div b$
$\lvert z \rvert$	
$\overline{z}$	

Complejo
- real: double - imaginaria: double - static final double EPSILON: double
+ Complejo() + Complejo(double real, double imaginaria) + getReal(): double + getImaginaria(): double + setReal(double real): void + setImaginaria(double imaginaria): void + sumar(Complejo otro):void + restar(Complejo otro):void + multiplicar(Complejo otro):void + dividir(Complejo otro):void + modulo(): double + argumento(): double + conjugado(): Complejo + equals(Object obj): boolean + toString(): String + static sumar(Complejo c1, Complejo c2): Complejo + static restar(Complejo c1, Complejo c2): Complejo + static multiplicar(Complejo c1, Complejo c2): Complejo + static dividir(Complejo c1, Complejo c2): Complejo

Deben estar presentes los métodos constructores así como los de acceso a los atributos.

Incluir métodos útiles adicionales como:

- **modulo()**: calcula $|z| = \sqrt{(a^2 + b^2)}$
- **argumento()**: calcula $\theta = \arctan(b/a)$
- **conjugado()**: retorna el conjugado (a - bi)
- **opuesto():** retorna el opuesto (-a - bi)
- **potencia(int n)**: calcula z^n
- **equals(Object otro)**: compara igualdad con tolerancia
- **toString()**: representación en formato a+bi

En el método **main**, instanciar varios objetos de la clase Complejo y probar su funcionamiento.

Ejercicio 3. Desarrolle una implementación en Java de la clase **Racional** que permita representar y operar con números racionales (fracciones). La implementación debe cumplir con los siguientes requisitos:

- La clase debe tener atributos para almacenar el numerador y denominador y debe implementar operaciones básicas como **métodos de instancia**: Suma, Resta, Multiplicación y División.

- Las mismas operaciones se debe proporcionar como **métodos estáticos** que reciben dos objetos Racional y retornan un nuevo objeto con el resultado.

Incluir métodos útiles para el manejo de números racionales en aplicaciones prácticas:

- Simplificación de fracciones
- Conversión a decimal
- Comparación de racionales
- Representación como cadena de texto

En el método **main**, instanciar varios objetos de la clase Racional y probar su funcionamiento.

Racional
- numerador: int - denominador: int
+ Racional() + Racional(int numerador, int denominador) + getNumerador(): int + getDenominador(): int + setNumerador(int numerador): void + setDenominador(int denominador): void + sumar(Racional r):void + restar(Racional r):void + multiplicar(Racional r):void + dividir(Racional r):void + simplificar(): Racional + aDecimal(): double + toString(): String + equals(Object obj): boolean + compareTo(Racional r): int + static sumar(Racional r1, Racional r2): Racional + static restar(Racional r1, Racional r2): Racional + static multiplicar(Racional r1, Racional r2): Racional + static dividir(Racional r1, Racional r2): Racional - static mcd(int a, int b): int

Ejercicio 4. Codifica una clase **Polinomio** que sirva para el manejo de polinomios matemáticos con las siguientes especificaciones:

- Un atributo **grado** de tipo entero
- Un arreglo **coeficientes** de tipo double para almacenar los coeficientes donde el índice representa la potencia de x

Proporcionar los siguientes métodos:

- Constructor que recibe un arreglo de coeficientes
- Constructor que recibe un grado y crea un polinomio con todos los coeficientes en cero
- Métodos de **instancia** para operaciones básicas que modifican los atributos del Polinomio:
  - ➔ **sumar(Polinomio p)**: Suma dos polinomios
  - ➔ **restar(Polinomio p)**: Resta un polinomio del actual
  - ➔ **multiplicar(Polinomio p)**: Multiplica dos polinomios
  - ➔ **dividir(Polinomio p)**: Divide el polinomio actual por otro y devuelve el cociente y el residuo como un arreglo

- Métodos **estáticos** que realizan las operaciones y devuelven el resultado en otro objeto:
  - **sumar(Polinomio p1, Polinomio p2)**
  - **restar(Polinomio p1, Polinomio p2)**
  - **multiplicar(Polinomio p1, Polinomio p2)**
  - **dividir(Polinomio p1, Polinomio p2)**
- Métodos adicionales útiles:
  - **evaluar(double x)**: Evalúa el polinomio en un punto usando el algoritmo de Horner
  - **derivar()**: Calcula la derivada del polinomio
  - **integrar()**: Calcula la integral indefinida del polinomio
  - **calcularRaices()**: Calcula las raíces para polinomios de grado ≤ 2
  - **toString()**: Convierte el polinomio a una representación de cadena en formato matemático

Polinomio
- grado: int - coeficientes: double[]
+ Polinomio(coeficientes: double[]) + Polinomio(grado: int) + getGrado(): int + getCoeficientes(): double[] + getCoeficiente(i: int): double + setCoeficiente(i: int, c: double) + ingresarPolinomio(): void + agregarCoeficiente(coef: double, exp: int): void + sumar(p: Polinomio):void + restar(p: Polinomio):void + multiplicar(p: Polinomio):void + dividir(p: Polinomio): Polinomio[] + evaluar(x: double): double + derivar(): Polinomio + integrar(): Polinomio + calcularRaices(): double[] + toString(): String + static sumar(p1: Polinomio, p2: Polinomio): Polinomio + static restar(p1: Polinomio, p2: Polinomio): Polinomio + static multiplicar(p1: Polinomio, p2: Polinomio): Polinomio + static dividir(p1: Polinomio, p2: Polinomio): Polinomio[]

Ejercicio 5. Codifica una clase Polinomio que sirva para el manejo de Conjuntos matemáticos con las siguientes especificaciones: